

V1.

Эта версия архитектуры максимально близка к той, что была использована в гл. [5.2 Создаём свою архитектуру](#).

Есть всё же некоторые изменения:

- Оpcodes инструкций пронумерованы заново, их можно увидеть в файле [instr.h](#)
- Добавлена инструкция NOP, декодер игнорирует ее при распаковке кода.

Потребность в такой инструкции возникает из-за упаковки кода Vuint7, при упаковке инструкций GOTO и CALL адрес перехода может быть еще неизвестен, так что мы резервируем место под максимальный адрес (в данном случае 2 байта) с помощью инструкции NOP. В дальнейшем, когда адрес перехода определится, запишем его в зарезервированное место и если он окажется короче, то остаток будет проигнорирован декодером.

- Вместо набора инструкций сравнения (*i eq, i gt, i ne, ...*) использована единая инструкция *i_cmp* с аргументом - типом сравнения. Аргумент это битовая маска из трёх разрядов. Первый (младший) разряд отвечает за “меньше”, средний за “равенство” и третий за “больше”. Так, старая инструкция *i_ge* эквивалентна *cmp 6*.

Такой набор инструкций упрощает внутреннее устройство процессора, хотя и ценой распухшего кода, ведь инструкции из однобайтовых стали двухбайтовыми. Впрочем, сейчас нам важнее простота, возможно, в дальнейшем еще вернёмся к этому вопросу.

- Адреса начала стека, начала первого фрейма и стартовая позиция кода теперь не зашиты жестко, а определяются при компиляции/сборке программы

Ассемблер

Задача первой версии ассемблера - перейти от программирования в автокоде к языку ассемблера, где используются мнемоники инструкций вместо opcodes и имена меток вместо абсолютных адресов. Одно это экономит программисту кучу времени в основном из-за того, что избавляет от кучи обидных оцепяток. Особенно учитывая то, что в нашей архитектуре код упаковывается Vuint7.

Исходный код ассемблера находится по адресу [zasm](#).

Ассемблер написан с использованием генераторов синтаксического (bison) и лексического (flex) анализаторов, что может показаться избыточным для такой, в общем-то, несложной задачи. Но в дальнейшем это сэкономит нам кучу времени и сил.

Использованная грамматика:

```
%start section_list

%%
section_list
```

```

    : section
    | section_list section

section
    :
    | data_section
    | ccode_section
    | xcode_section

data_section
    : ZASM_DATA_SECTION '{' data_list '}'

data_list
    : data_item
    | data_list data_item

data_item
    : ZASM_IDENTIFIER ':' data_elem_list ';'

data_elem_list
    : data_elem
    | data_elem_list data_elem

data_elem
    : ZASM_INTEGER_LITERAL

ccode_section
    : ZASM_CCODE_SECTION
    '{' code_list '}'

code_list
    : code_item
    | code_list ccode_item

code_item
    : ZASM_IDENTIFIER ':'
    | instr_item

xcode_section
    : ZASM_XCODE_SECTION
    '{'code_list '}'

instr_item
    : ZASM_INSTR instr_args ';'
    | ZASM_INSTR ';'

instr_args
    : instr_arg
    | instr_args instr_arg

instr_arg
    : ZASM_FLOAT_LITERAL
    | ZASM_IDENTIFIER
    | ZASM_STRING_LITERAL
```

| ZASM_INTEGER_LITERAL

Рассмотрим как это выглядит на практике.

Возьмем уже использовавшийся нами пример вычисления факториала:

```
int factorial(int i)
{
    if (i==0) return 1;
    else return i*factorial(i-1);
}
...
factorial(6);
```

На языке ассемблера это будет выглядеть так:

```
DATA {
    a: 0x0001
    b: 0x0002
    c: 0x0003
    d: 0x0004
}
```

Для начала секция глобальных данных. В примере таковые не использовались, но фактически существовали, так что создаем секцию. В ней описаны четыре переменные каждая размером в 32 разряда и определены их значения. В дальнейшем потребуется секция константных данных, назовем её CDATA. Секций таких может быть несколько, они могут перемежаться с кодом.

```
CCODE {
initial:
    exec block1;
    call factorial 4 0;
}
CCODE {
factorial:
    exec block2;
    if endif1;
    exec block3;
    ret;
endif1:
    exec block4;
    call factorial 4 0
    exec block5;
    ret;
}
```

Секции типа CCODE содержат код потока управления. В них могут быть инструкции потока управления и именованные метки. На метку можно перейти инструкцией `goto` или попасть через вызов функции. Если блок не заканчивается инструкцией `stop`, она добавляется автоматически.

```
XCODE {
block1:
    imdpush 6;
    stop;

block2:
    imdpush 1;
    locpush 0;
    eval;
    eq;
    stop;

block3:
    imdpush 1;
    stop;

block4:
    lockpush 0;
    eval;
    imdpush 1;
    sub;
    stop;

block5:
    locpush 0;
    eval;
    mul;
    stop;
}
```

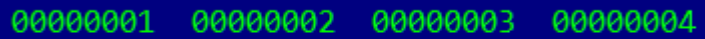
Секция потока исполнения. Их может быть несколько и вполне могут перемежаться с секциями других типов. Содержит набор именованных блоков, по которым к ним обращаются инструкции потока управления.

Собираем `zasm`, запускаем (`zasm -f test.asm`), на выходе получаем два файла с образами памяти: данных (`data_segment.mem`) и кода (`rom_image.mem`), аналогичных тем, что мы использовали в главе [5.2 Создаём свою архитектуру](#).

Кроме того, ассемблер сообщает нам размер секции глобальных данных - начало стека и первого фрейма, а также начальный адрес для исполнения - в ассемблере ему соответствует метка `"initial"`.

```
Initial code address: 0x1a
Stack start position: 0x10
```

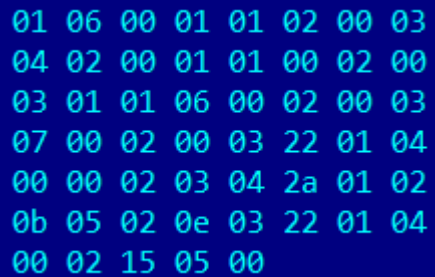
Фиг.5.3.1 Адреса в выдаче ассемблера



```
00000001 00000002 00000003 00000004
```

Фиг.5.3.3. Содержимое data_segment.mem

Сегмент данных - четыре слова с глобальными переменными, далее неинициализированная память под стек.



```
01 06 00 01 01 02 00 03
04 02 00 01 01 00 02 00
03 01 01 06 00 02 00 03
07 00 02 00 03 22 01 04
00 00 02 03 04 2a 01 02
0b 05 02 0e 03 22 01 04
00 02 15 05 00
```

Фиг.5.3.3. Содержимое rom_image.mem.

Сегмент кода - бинарное представление для нашей архитектуры версии V1.

“Пилоты молчат, смотрят на звёзды.

Задание выполнено ... успешно.

Все улыбаются.” (C)